In [33]:
```python
"""INF201 Innlevering 2"""

import json
import re
from pathlib import Path


def read_log_file(path: str | Path) -> tuple[str, str]:

    path = Path(path)
    raw = path.read_bytes()

    try:
        return raw.decode("utf-8"), "utf-8"
    except UnicodeDecodeError:
        return raw.decode("iso-8859-1"), "iso-8859-1"



LOG_PATTERN = re.compile(
    r"^\[(?P<timestamp>\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2})\]\s+"
    r"\[(?P<register_id>[^\]]+)\]\s+"
    r"item=(?P<item>.+?)\s+"
    r"price=(?P<price>-?\d+(?:\.\d+)?)\s+"
    r"(?P<currency>[A-Z]{3})\s+"
    r"\((?P<status>SUCCESS|REFUND|CANCELLED)\)\s*$",
    re.MULTILINE,
)


def parse_log_text(text: str) -> list[dict]:

    records = []
    for match in LOG_PATTERN.finditer(text):
        record = match.groupdict()
        record["price"] = float(record["price"])
        records.append(record)
    return records



def find_log_files(directory: str | Path, extension: str = ".log") -> lis
    """Finner rekursivt alle filer med gitt filendelse."""
    directory = Path(directory)
    found: list[Path] = []

    for item in directory.iterdir():
        if item.is_dir():
            found.extend(find_log_files(item, extension))
        elif item.is_file() and item.suffix == extension:
            found.append(item)

    return sorted(found)



def get_active_register_ids(node) -> set[str]:
    """Returnerer id-ene til alle registre med "active": true.
```

```
    Fungerer på enhver kombinasjon av nøstede dictionaries og lister.
    """
    active: set[str] = set()

    if isinstance(node, dict):
        if node.get("active") is True and isinstance(node.get("id"), str)
            active.add(node["id"])
        for value in node.values():
            active |= get_active_register_ids(value)
    elif isinstance(node, list):
        for element in node:
            active |= get_active_register_ids(element)

    return active
```

Task 3: Rydding og analyse med NumPy

remove_price_outliers: gjør prisene om til en 1D NumPy-array og regner ut gjennomsnittet og standardavvik. priser der (x - μ) er større enn 3 standardavvik regnes som feil (altså som 999.00), og byttes ut med medianen av de gyldige prisene. vi bruker medianen fordi den ikke påvirkes av ekstreme verider.

In [34]:
```
#compute_refund_ratio: regner ut refusjonsandelen element for element med

import numpy as np

def remove_price_outliers(prices) -> np.ndarray:
    "Replace outliers (|x - mean| > 3*std) with the median of the valid p
    prices = np.asarray(prices, dtype=float).ravel()  #1D float array

    mean = prices.mean()
    std = prices.std()
    is_outlier = np.abs(prices - mean) > 3 * std

    cleaned = prices.copy()
    if is_outlier.any():
        cleaned[is_outlier] = np.median(prices[~is_outlier])
    return cleaned
def compute_refund_ratio(sales_count: np.ndarray, refund_count: np.ndarra
    """Element-wise refund ratio. Epsilon avoids division by zero."""
    return refund_count / (sales_count + 1e-6)
```

Rask test av funksjonene med egne tall: 999.0 skal byttes ut med medianen

In [35]:
```
#Tester med tilfeldige numre
test_prices = [4.5, 5.0, 4.0, 5.5, 4.5, 5.0, 4.0, 5.5, 4.5, 5.0, 4.0, 5.5
print("Before:", test_prices)
print("After: ", remove_price_outliers(test_prices))
print("Refund ratio:", compute_refund_ratio(np.array([10, 0, 4]), np.arra
```

```
Before: [4.5, 5.0, 4.0, 5.5, 4.5, 5.0, 4.0, 5.5, 4.5, 5.0, 4.0, 5.5, 999.
0]
After:  [4.5  5.   4.   5.5  4.5  5.   4.   5.5  4.5  5.   4.   5.5  4.75]
Refund ratio: [0.19999998 0.         0.24999994]
```

Task 4: Dataanalyse med pandas

build_dataframe: gjør listen med transaksjoner om til en dataframe, gjør "timestamp" om til en datetime, setter den som indeks og sorterer kronologisk

hourly_revenue_per_item: grupperer på vare og summerer prisene i vinduer på 1 time med "resample"("1h")

status_pivot_table: lager en pivottabelk med varer som rader, status (SUCCESS, REFUND, CANCELLED) som kolonner og total omsetning i cellene

refund_ratio_per_item: teller antall salg (SUCCESS) og refusjoner (REFUND) per vare, og bruker "compute_refund_ratio" fra Task 3

In [41]:
```
import pandas as pd

def build_dataframe(records: list[dict]) -> pd.DataFrame:
    """List of transaction dicts -> DataFrame with timestamp as sorted in
    df = pd.DataFrame(records)
```

```
    df["timestamp"] = pd.to_datetime(df["timestamp"])
    df = df.set_index("timestamp").sort_index()
    return df

def hourly_revenue_per_item(df: pd.DataFrame) -> pd.DataFrame:
    """Total price per item in 1-hour windows."""
    hourly = df.groupby("item")["price"].resample("1h").sum()
    return hourly.reset_index()  #colums: item, timestamp, price

def status_pivot_table(df: pd.DataFrame) -> pd.DataFrame:
    """Total revenue per item for each status."""
    return pd.pivot_table(
        df,
        values="price",
        index="item",
        columns="status",
        aggfunc="sum",
        fill_value=0,
    )

def refund_ratio_per_item(df: pd.DataFrame) -> pd.DataFrame:
    """Refund ratio per item = number of refund / number of successful sa
    counts = df.groupby(["item", "status"]).size().unstack(fill_value=0)
    sales = counts.get("SUCCESS", pd.Series(0, index=counts.index)).to_nu
    refunds = counts.get("REFUND", pd.Series(0, index=counts.index)).to_n

    result = pd.DataFrame({
        "sales_count": sales,
        "refund_count": refunds,
        "refund_ratio": compute_refund_ratio(sales, refunds),
    }, index=counts.index)
    return result
```

Task 5: Hele arbeidsflyten

Her setter vi sammen funksjonene fra Task 1-4:

1. Finner alle loggfiler rekursivt og leser aktive kasser fra "store_config.json".

2. Leser filene (UTF-8 eller Latin-1) og henter ut gyldige salgslinjer.

3. Lager en DataFrame. vi tar bare med transaksjoner fra aktive kasser.

4. fjerner 999.00-feilene med NumPy.

5. retter NOK-priser som var registrert i EUR-skala, ved å gange med 10.

6. lager rapportene

7. lagrer de fire output-filene

In [42]:
```python
import json
from pathlib import Path

DATA_DIR = Path("data")
CONFIG_FILE = DATA_DIR / "store_config.json"

#1. Find all log files and the active register IDs
log_files = find_log_files(DATA_DIR)
config = json.loads(CONFIG_FILE.read_text(encoding="utf-8"))
active_registers = get_active_register_ids(config)
print(f"Found {len(log_files)} log files")
print(f"Active registers: {sorted(active_registers)}")

#2. Read every file and parse the valid sales lines
records = []
for path in log_files:
    text, encoding = read_log_file(path)
    records.extend(parse_log_text(text))
    print(f"Parsed {len(records)} transactions")

    #keep only transactionw from active registers
    records = [r for r in records if r["register_id"] in active_registers
    print(f"{len(records)} transactions from active registers")

    #3. Build the DataFrame
    df = build_dataframe(records)

    #4. remove 999.00 price glitches
    n_glitches = (df["price"] >= 999).sum()
    df["price"] = remove_price_outliers(df["price"])
    print(f"Replaced {n_glitches} outliers")

    #5. fix NOK prices that were registered on EUR scale
    df.loc[df["currency"] == "NOK", "price"] *= 10.0
    df["price"] = df ["price"].round(2)

    #6. reports
    hourly = hourly_revenue_per_item(df)
    pivot = status_pivot_table(df)
    refunds = refund_ratio_per_item(df)

    #7. save output files
    df.to_csv("cleaned_transactions.csv")
```

```
    hourly_json = {}
    for item, group in hourly.groupby("item"):
        hourly_json[item] = {
            ts.strftime("%Y-%m-%d %H: %M"): round(float(price), 2)
            for ts, price in zip(group["timestamp"], group["price"])

        }
    Path("hourly_revenue.json").write_text(json.dumps(hourly_json, indent
    )

    pivot.to_csv("status_pivot_table.csv")
    refunds.to_csv("item_refund_ratios.csv")

    print("\nSaved: cleaned_transactions.csv, hourly_revenue.json, "
          "status_pivot_table.csv, item_refund_ratios.csv")

    print("Pivot table (total revenue per item and status):")
    print(pivot.round(2))
    print("\nRefund ratio per item:")
    print(refunds.round(3))
    df.head()
```

```
Found 9 log files
Active registers: ['POS_01', 'POS_02', 'POS_03']
Parsed 80 transactions
80 transactions from active registers
Replaced 4 outliers

Saved: cleaned_transactions.csv, hourly_revenue.json, status_pivot_table.c
sv, item_refund_ratios.csv
Pivot table (total revenue per item and status):
status      CANCELLED  REFUND  SUCCESS
item
Cappuccino      46.46   54.74   207.80
Croissant       38.13   18.89    53.29
Espresso         0.00   57.50    75.40
Latte            9.61    9.53   190.91
Muffin          39.36   44.77   124.73
Sandwich        87.10    0.00   314.88

Refund ratio per item:
            sales_count  refund_count  refund_ratio
item
Cappuccino           14             3         0.214
Croissant             5             5         1.000
Espresso              7             2         0.286
Latte                 8             2         0.250
Muffin               10             3         0.300
Sandwich             10             0         0.000
Parsed 160 transactions
160 transactions from active registers
Replaced 8 outliers

Saved: cleaned_transactions.csv, hourly_revenue.json, status_pivot_table.c
sv, item_refund_ratios.csv
Pivot table (total revenue per item and status):
status      CANCELLED  REFUND  SUCCESS
item
Cappuccino      58.06   63.30   399.27
Croissant       53.49   30.28   117.73
Espresso        33.30   63.38   128.31
Latte           18.30   13.73   332.51
Muffin         123.56   51.77   244.94
Sandwich        94.84    7.56   363.70

Refund ratio per item:
            sales_count  refund_count  refund_ratio
item
Cappuccino           25             5         0.200
Croissant            13             8         0.615
Espresso             15             4         0.267
Latte                20             3         0.150
Muffin               18             5         0.278
Sandwich             17             1         0.059
Parsed 240 transactions
240 transactions from active registers
Replaced 13 outliers

Saved: cleaned_transactions.csv, hourly_revenue.json, status_pivot_table.c
sv, item_refund_ratios.csv
Pivot table (total revenue per item and status):
status      CANCELLED  REFUND  SUCCESS
```

```
item
Cappuccino      65.96  121.80   528.31
Croissant       60.54  142.71   224.33
Espresso        33.30   63.38   245.43
Latte           27.55   63.43   588.59
Muffin         159.91   54.95   331.27
Sandwich       109.35   86.52   777.32

Refund ratio per item:
            sales_count  refund_count  refund_ratio
item
Cappuccino           38             9         0.237
Croissant            23            12         0.522
Espresso             21             4         0.190
Latte                31             5         0.161
Muffin               26             6         0.231
Sandwich             27             3         0.111
Parsed 320 transactions
320 transactions from active registers
Replaced 16 outliers

Saved: cleaned_transactions.csv, hourly_revenue.json, status_pivot_table.c
sv, item_refund_ratios.csv
Pivot table (total revenue per item and status):
status      CANCELLED  REFUND  SUCCESS
item
Cappuccino      78.98  173.68   682.75
Croissant      163.86  216.23   312.09
Espresso        75.68   96.75   410.79
Latte           27.55   76.89   674.51
Muffin         236.92   58.04   380.97
Sandwich       186.25   86.52   884.60

Refund ratio per item:
            sales_count  refund_count  refund_ratio
item
Cappuccino           46            13         0.283
Croissant            29            14         0.483
Espresso             34             6         0.176
Latte                41             8         0.195
Muffin               30             7         0.233
Sandwich             32             3         0.094
Parsed 400 transactions
400 transactions from active registers
Replaced 16 outliers

Saved: cleaned_transactions.csv, hourly_revenue.json, status_pivot_table.c
sv, item_refund_ratios.csv
Pivot table (total revenue per item and status):
status      CANCELLED  REFUND  SUCCESS
item
Cappuccino      83.18  177.63   864.42
Croissant      240.76  286.54   498.39
Espresso        78.89  109.13   655.05
Latte           72.58   81.13   928.77
Muffin         274.67   61.40   576.37
Sandwich       333.45  101.16  1182.07

Refund ratio per item:
            sales_count  refund_count  refund_ratio
```

```
item
Cappuccino           55            14         0.255
Croissant            35            16         0.457
Espresso             44            10         0.227
Latte                54             9         0.167
Muffin               38             8         0.211
Sandwich             44             5         0.114
Parsed 480 transactions
480 transactions from active registers
Replaced 16 outliers

Saved: cleaned_transactions.csv, hourly_revenue.json, status_pivot_table.c
sv, item_refund_ratios.csv
Pivot table (total revenue per item and status):
status      CANCELLED  REFUND  SUCCESS
item
Cappuccino     125.78  232.71  1008.80
Croissant      240.76  324.00   682.24
Espresso       151.58  145.68   769.11
Latte          130.40  132.24  1032.39
Muffin         346.89   65.35   727.82
Sandwich       490.94  257.42  1375.96

Refund ratio per item:
            sales_count  refund_count  refund_ratio
item
Cappuccino           64            17         0.266
Croissant            45            18         0.400
Espresso             53            12         0.226
Latte                60            11         0.183
Muffin               47             9         0.191
Sandwich             52             9         0.173
Parsed 560 transactions
560 transactions from active registers
Replaced 19 outliers

Saved: cleaned_transactions.csv, hourly_revenue.json, status_pivot_table.c
sv, item_refund_ratios.csv
Pivot table (total revenue per item and status):
status      CANCELLED  REFUND  SUCCESS
item
Cappuccino     255.42  232.71  1201.97
Croissant      356.06  365.17   842.88
Espresso       210.10  145.68   780.83
Latte          176.80  140.54  1277.26
Muffin         361.08  211.75   827.32
Sandwich       598.27  257.42  1540.00

Refund ratio per item:
            sales_count  refund_count  refund_ratio
item
Cappuccino           74            17         0.230
Croissant            58            21         0.362
Espresso             57            12         0.211
Latte                69            13         0.188
Muffin               57            14         0.246
Sandwich             56             9         0.161
Parsed 640 transactions
640 transactions from active registers
Replaced 22 outliers
```

```
Saved: cleaned_transactions.csv, hourly_revenue.json, status_pivot_table.c
sv, item_refund_ratios.csv
Pivot table (total revenue per item and status):
status      CANCELLED  REFUND  SUCCESS
item
Cappuccino     310.15  249.97  1347.10
Croissant      359.71  372.30  1062.24
Espresso       238.20  152.12   835.84
Latte          181.11  144.92  1519.06
Muffin         374.98  256.32   889.46
Sandwich       677.54  343.23  1730.68

Refund ratio per item:
            sales_count  refund_count  refund_ratio
item
Cappuccino           81            21         0.259
Croissant            67            23         0.343
Espresso             66            14         0.212
Latte                78            14         0.179
Muffin               66            18         0.273
Sandwich             64            11         0.172
Parsed 720 transactions
720 transactions from active registers
Replaced 25 outliers

Saved: cleaned_transactions.csv, hourly_revenue.json, status_pivot_table.c
sv, item_refund_ratios.csv
Pivot table (total revenue per item and status):
status      CANCELLED  REFUND  SUCCESS
item
Cappuccino     400.95  300.98  1604.72
Croissant      374.54  372.30  1180.43
Espresso       238.20  178.32   911.51
Latte          231.28  144.92  1719.37
Muffin         489.24  259.37  1011.58
Sandwich       677.54  472.19  2092.87

Refund ratio per item:
            sales_count  refund_count  refund_ratio
item
Cappuccino           91            23         0.253
Croissant            80            23         0.287
Espresso             74            15         0.203
Latte                86            14         0.163
Muffin               74            19         0.257
Sandwich             75            15         0.200
```

In [43]:

```python
from pathlib import Path

gen_fil = next(Path(".").glob("Generate*Deliverable*2*.py"))
navnerom = {}
exec(gen_fil.read_text(encoding="utf-8"), navnerom)
navnerom["BASE_DIR"] = Path("data")
navnerom["create_dataset"]()
```

```
Generated: data/2026/library_cafe/POS_01/register_sales.log [UTF-8]
Generated: data/2026/library_cafe/POS_02/register_sales.log [ISO-8859-1]
Generated: data/2026/library_cafe/POS_03/register_sales.log [UTF-8]
Generated: data/2026/science_building/POS_01/register_sales.log [ISO-8859-
1]
Generated: data/2026/science_building/POS_02/register_sales.log [UTF-8]
Generated: data/2026/science_building/POS_03/register_sales.log [ISO-8859-
1]
Generated: data/2026/student_union/POS_01/register_sales.log [UTF-8]
Generated: data/2026/student_union/POS_02/register_sales.log [ISO-8859-1]
Generated: data/2026/student_union/POS_03/register_sales.log [UTF-8]

Dataset generation complete!
```

In [ ]:

In [ ]: